

Practical 1

Aim: To implement contour detection for identifying boundaries of shapes and classifying different types of shapes in an image.

Theory: Contours are fundamental tools in image processing used to detect object boundaries by identifying curves that connect points of equal intensity. Typically, contour detection begins with preprocessing steps such as converting the image to grayscale, applying noise reduction techniques like Gaussian blur, and performing thresholding or edge detection (e.g., Canny edge detector). Once edges are highlighted, functions such as **findContours()** in OpenCV are used to retrieve contour information. These contours can then be approximated using algorithms like **approxPolyDP()** to simplify complex shapes into polygons. By analysing features such as the number of vertices, perimeter, area, and aspect ratio, different geometric shapes like triangles, rectangles, and circles can be classified. Contour hierarchy also helps in understanding nested structures within images. This technique is computationally efficient and widely applied in object detection, shape analysis, motion tracking, and pattern recognition tasks in computer vision systems.

Code: To be printed

Conclusion: Contour detection effectively identifies object boundaries and enables accurate shape classification, making it a fundamental technique in image processing and computer vision applications.

Practical 2

Aim:

To implement graph cut segmentation

Theory: Graph cut segmentation is an advanced image segmentation technique that models an image as a weighted graph to partition it into meaningful regions. Each pixel is represented as a node, and edges are formed between neighbouring pixels, with weights reflecting similarity based on features such as colour, intensity, or texture. Additionally, two special nodes called the source (foreground) and sink (background) are introduced. The segmentation task is formulated as an energy minimization problem, where the objective is to minimize a cost function that balances two factors: data fidelity (how well pixels match the assigned label) and smoothness (encouraging similar neighbouring pixels to belong to the same segment). Algorithms such as min-cut/max-flow are used to compute the optimal partition of the graph. The resulting cut effectively separates the image into foreground and background regions. This method is highly effective for complex images and is widely used in medical imaging, object extraction, and interactive segmentation applications.

Code: To be printed

Conclusion:

Graph cut segmentation provides accurate and efficient separation of foreground and background by optimizing energy functions, making it suitable for complex segmentation tasks.

Practical 3

Aim:

To implement mean shift segmentation

Theory: Mean shift segmentation is a non-parametric clustering technique widely used for image segmentation based on feature space analysis. In this approach, each pixel is treated as a data point in a multidimensional feature space that typically includes spatial coordinates and colour values. The algorithm works by iteratively shifting each data point toward the region of highest data density, known as the mode, using a kernel function such as a Gaussian kernel. This shifting process continues until convergence, meaning that points stabilize around local maxima. Pixels that converge to the same mode are grouped into a cluster, forming a segmented region in the image. Unlike methods such as k-means, mean shift does not require prior knowledge of the number of clusters, making it more flexible. It effectively preserves edges while smoothing regions, which helps in handling noise and variations in intensity. This technique is particularly useful in applications like object tracking, image smoothing, and feature space analysis in computer vision.

Code: To be printed

Conclusion:

Mean shift segmentation effectively groups similar pixels into regions without prior assumptions, providing smooth and edge-preserving image segmentation results.

Practical 4

Aim: To obtain a histogram equalization image

Theory: Histogram equalization is a technique used in image processing to enhance contrast by redistributing the intensity values of an image. In many images, especially those that are underexposed or overexposed, pixel intensities are concentrated within a narrow range, resulting in poor contrast. Histogram equalization works by spreading out these intensity values across the entire available range, typically 0 to 255 for grayscale images. This is achieved by computing the cumulative distribution function (CDF) of the image histogram and mapping the original pixel values to new ones based on this distribution. As a result, areas with low contrast gain improved visibility, and finer details become more distinguishable. The method is particularly useful in applications such as medical imaging, satellite imagery, and low-light photography. While effective, it may sometimes amplify noise or over-enhance certain regions if not applied carefully.

Code: To be printed

Conclusion: Histogram equalization improves image contrast and enhances visibility of details, making it a useful technique for image enhancement in various applications.

Practical 5

Aim: To implement a colour detection and edge detection algorithm

Theory: Colour detection and edge detection are essential techniques in image processing used for identifying objects and their boundaries. Colour detection involves identifying specific colors within an image, typically using color spaces such as RGB or HSV. The HSV color space is often preferred as it separates color information (hue) from intensity, making detection more robust under varying lighting conditions. Thresholding is applied to isolate pixels within a specific color range, resulting in a mask that highlights the desired color regions. Edge detection, on the other hand, focuses on identifying sharp changes in intensity, which correspond to object boundaries. Algorithms such as the Canny edge detector are widely used, involving steps like noise reduction, gradient calculation, non-maximum suppression, and hysteresis thresholding. Combining both techniques allows for precise object recognition and boundary extraction. These methods are widely used in applications such as object tracking, robotics, and computer vision systems.

Code: To be printed

Conclusion: Colour and edge detection together enable effective identification and localization of objects, playing a crucial role in modern computer vision applications.

Practical 6

Aim:

To implement Object tracking with particle filters

Theory: Particle filter-based object tracking is a probabilistic approach used to estimate the state of a moving object over time in a sequence of images or video frames. It represents the object's state (such as position, velocity, or scale) using a set of random samples called particles. Each particle is assigned a weight based on how well it matches the observed data, typically using features like color, texture, or shape. The algorithm follows three main steps: prediction (particles are propagated based on a motion model), update (weights are assigned based on observation likelihood), and resampling (particles with higher weights are retained while others are discarded). This iterative process allows the filter to adapt to changes such as noise, occlusion, or dynamic backgrounds. Particle filters are particularly effective for non-linear and non-Gaussian tracking problems, making them widely applicable in surveillance, robotics, and human-computer interaction systems.

Code: To be printed

Conclusion:

Particle filters provide a robust and flexible method for tracking objects in dynamic environments, handling uncertainty and complex motion effectively.

Practical 7

Aim: To implement the hand gesture recognition

Theory: Hand gesture recognition is a computer vision technique used to interpret human hand movements and gestures for interaction with machines. The process typically begins with image acquisition followed by preprocessing steps such as background subtraction, noise removal, and skin color detection using color spaces like HSV or YCbCr. After isolating the hand region, techniques such as contour detection, convex hull, and convexity defects are used to extract features like finger positions and hand shape. These features are then analyzed or fed into machine learning models for classification of different gestures. Advanced systems may use deep learning approaches like convolutional neural networks (CNNs) for higher accuracy and real-time performance. Gesture recognition enables touchless interaction and is widely used in applications such as gaming, virtual reality, sign language interpretation, and human-computer interfaces.

Code: To be printed

Conclusion:

Hand gesture recognition enables intuitive and contactless interaction with systems, making it a key technology in modern human-computer interaction.